

Memory model synchronization modes

This is the area most people find confusing when looking at the memory model. **Atomic variables are primarily used to synchronize shared memory accesses between threads.** Typically one thread creates data, then stores to an atomic. Other threads read from this atomic, and when the expected value is seen, the data the other thread was creating is going to be complete and visible in this thread. The different memory model modes are used to indicate how strong this data-sharing bond is between threads. Knowledgeable programmers can utilize the weaker models to make more efficient software.

Each atomic class has a `load()` and a `store()` operation which is utilized to perform assignments. This helps make it clearer when atomic operations are being performed rather than a normal assignment.

```
atomic_var1.store (atomic_var2.load()); // atomic variables
vs
var1 = var2;    // regular variables
```

These operations also have a second optional parameter which is used to specify the memory model mode to use for synchronization.

There are 3 modes or models which allow the programmer to specify the type of inter-thread synchronization.

Sequentially Consistent

The first model is “sequentially consistent”. This is the default mode used when none is specified, and it is the most restrictive. It can also be explicitly specified via **`std::memory_order_seq_cst`**. It provides the same restrictions and limitation to moving loads around that sequential programmers are inherently familiar with, except it applies across threads.

```
-Thread 1-      -Thread 2-
y = 1           if (x.load() == 2)
x.store (2);    assert (y == 1)
```

Although `x` and `y` are unrelated variables, the memory model specifies that the `assert` cannot fail. The store to `'y'` *happens-before* the store to `x` in thread 1. If the load of `'x'` in thread 2 gets the results of the store that happened in thread 1, it must all see **all** operations that *happened before* the store in thread 1, even unrelated ones. That means the optimizer is not free to reorder the two stores in thread 1 since thread 2 **must** see the store to `Y` as well.

This also applies to loads as well:

```
        a = 0
        y = 0
        b = 1

-Thread 1-      -Thread 2-
x = a.load()    while (y.load() != b)
y.store (b)      ;
while (a.load() == x)  a.store(1)
                    ;
```

Thread 2 loops until the value of 'y' changes, and proceeds to change 'a'. thread 1 is waiting for 'a' to change.

When normally compiling sequential code, the 'while (a.load() == x)' in thread 1 looks like an infinite loop, and might be optimized as such. Instead, the load of 'a' and comparison to 'x' must happen on each iteration of the loop in order for thread 1 and 2 to proceed as expected.

From a practical point of view, this amounts to all atomic operations acting as optimization barriers. If one could imagine atomic loads and stores as function calls with unknown side effects, this would model the effects in the optimizers. Its OK to re-order things between atomic operations, but not across the operation. Thread local stuff is also unaffected since there is no visibility to other threads.

This mode also provides consistency across **all** threads. Neither assert in this example can fail (x and y are initially 0):

```
-Thread 1-      -Thread 2-      -Thread 3-
y.store (20);    if (x.load() == 10) {    if (y.load() == 10)
x.store (10);      assert (y.load() == 20)      assert (x.load() == 10)
                  y.store (10)
                  }
```

It seems reasonable to expect this behaviour, but cross thread implementation requires the system bus to be synchronized such that thread 3 actually gets the same results that thread 2 observed. This can involve some expensive hardware synchronization.

This mode is the default for good reason, programmers are far less likely to get unexpected results when this mode is in use.

Relaxed

The opposite approach is **std::memory_order_relaxed**. This model allows for much less synchronization by removing the *happens-before* restrictions. These types of atomic operations can also have various optimizations performed on them, such as dead store removal and commoning.

So in the earlier example:

```
-Thread 1-
y.store (20, memory_order_relaxed)
x.store (10, memory_order_relaxed)

-Thread 2-
if (x.load (memory_order_relaxed) == 10)
{
    assert (y.load(memory_order_relaxed) == 20) /* assert A */
    y.store (10, memory_order_relaxed)
}

-Thread 3-
if (y.load (memory_order_relaxed) == 10)
    assert (x.load(memory_order_relaxed) == 10) /* assert B */
```

Since threads don't need to be synchronized across the system, **either** assert in this example can actually **FAIL**.

Without any *happens-before* edges, no thread can count on a specific ordering from another thread. This can obviously lead to some unexpected results if one isn't very careful. The only ordering imposed is that once a value for a variable from thread 1 is observed in thread 2, thread 2 can not see an “earlier” value for that variable from thread 1. ie, assuming 'x' is initially 0:

```
-Thread 1-
x.store (1, memory_order_relaxed)
x.store (2, memory_order_relaxed)

-Thread 2-
y = x.load (memory_order_relaxed)
z = x.load (memory_order_relaxed)
assert (y <= z)
```

The assert cannot fail. Once the store of 2 is seen by thread 2, it can no longer see the value 1. This prevents coalescing relaxed loads of one variable across relaxed loads of a different reference that might alias.

There is also the presumption that relaxed stores from one thread are seen by relaxed loads in another thread within a reasonable amount of time. That means that on non-cache-coherent architectures, relaxed operations need to flush the cache (although these flushes can be merged across several relaxed operations)

The relaxed mode is most commonly used when the programmer simply wants an variable to be atomic in nature rather than using it to synchronize threads for other shared memory data.

Acquire/Release

The third mode is a hybrid between the other two. The acquire/release mode is similar to the sequentially consistent mode, except it only applies a *happens-before* relationship to **dependent** variables. This allows for a relaxing of the synchronization required between independent reads of independent writes.

Assuming 'x' and 'y' are initially 0:

```
-Thread 1-
y.store (20, memory_order_release);

-Thread 2-
x.store (10, memory_order_release);

-Thread 3-
assert (y.load (memory_order_acquire) == 20 && x.load
(memory_order_acquire) == 0)

-Thread 4-
assert (y.load (memory_order_acquire) == 0 && x.load (memory_order_acquire)
== 10)
```

Both of these asserts can pass since there is no ordering imposed between the stores in thread 1 and thread 2.

If this example were written using the sequentially consistent model, then one of the stores must *happen-before* the other (although the order isn't determined until run-time), the values are synchronized between threads, and if one assert passes, the other assert must therefore fail.

To make matters a bit more complex, the interactions of non-atomic variables are still the same. Any store before an atomic operation must be seen in other threads that synchronize. For example:

```
-Thread 1-
y = 20;
x.store (10, memory_order_release);

-Thread 2-
if (x.load(memory_order_acquire) == 10)
    assert (y == 20);
```

Since 'y' is not an atomic variable, the store to 'y' *happens-before* the store to 'x', so the assert cannot fail in this case. The optimizers must still limit the operations performed on shared memory variables around atomic operations.

Consume

std::memory_order_consume is a further subtle refinement in the release/acquire memory model that relaxes the requirements slightly by removing the happens before ordering on *non-dependent* shared variables as well.

Assuming n and m are both regular shared variables initially set to 0, and each thread picks up the store to 'p' in thread 1 :

```
-Thread 1-
n = 1
m = 1
p.store (&n, memory_order_release)

-Thread 2-
t = p.load (memory_order_acquire);
assert( *t == 1 && m == 1 );

-Thread 3-
t = p.load (memory_order_consume);
assert( *t == 1 && m == 1 );
```

The assert in thread 2 will pass because the store to 'm' *happens-before* the store to p in thread 1.

The assert in thread 3 **can** fail because there is no longer a dependency between the store to p, and the store to m, and therefore the values do not need to be synchronized. This models the PowerPC and ARM's default memory ordering for pointer loads. (and possibly some MIPS targets)

Both threads will see the correct value of `n == 1` since it is used in the store expression.

The real difference boils down to how much state the hardware has to flush in order to synchronize. Since a consume operation may therefore execute faster, someone who knows what they are doing can use it for performance critical applications.

Overall Summary

Its actually not as complex as it sounds, so in an attempt to un-glaze your eyes, examine this case for each of the different memory models:

```

-Thread 1-      -Thread 2-      -Thread 3-
y.store (20);    if (x.load() == 10) {    if (y.load() == 10)
x.store (10);    assert (y.load() == 20)    assert (x.load() == 10)
                y.store (10)
                }

```

When 2 threads synchronize in **sequentially consistent** mode, all the visible variables must be flushed through the system so that all threads see the same state. Both asserts must therefore be true.

Release/acquire mode only requires the two threads involved to be synchronized. This means that synchronized values are not commutative to other threads. The assert in thread 2 must still be true since thread 1 and 2 synchronize with `x.load()`. Thread 3 is not involved in this synchronization, so when thread 2 and 3 synchronize with `y.load()`, thread 3's assert **can** fail. There has been no synchronization between threads 1 and 3, so no value can be assumed for 'x' there.

If the stores are **release** and loads are **consume** the results are the same as release/acquire except there may be less hardware synchronization required. Why not always use consume? The reason is that this example doesn't have any shared memory being synchronized. You may not see the values of any shared memory before the stores in the synchronization unless it is a parameter to the store.. ie, its only a synchronization on shared memory variables used to calculate the store value.

If everything were **relaxed**, then both asserts can fail because there is no synchronization at all.

Mixing memory models

And finally, what about mixed modes... ie:

```

-Thread 1-
y.store (20, memory_order_relaxed)
x.store (10, memory_order_seq_cst)

-Thread 2-
if (x.load (memory_order_relaxed) == 10)
{
    assert (y.load(memory_order_seq_cst) == 20) /* assert A */
    y.store (10, memory_order_relaxed)
}

-Thread 3-
if (y.load (memory_order_acquire) == 10)
    assert (x.load(memory_order_acquire) == 10) /* assert B */

```

First off, **don't do it**, it will be very confusing! 😊

Second, Its still a fair question, so taking a stab at it... Think of what happens for synchronization at each point. Stores tend to execute the store, then execute whatever system flushing needs to be done on their processor. Loads will issue whatever synchronizing instructions are needed to acquire all the states that have been flushed, and then execute the load.

Thread 1 : `y.store` is relaxed, so it issues no synchronizations and can be moved around. `x.store` is `seq_cst`, so it forces the state of thread 1 to be flushed before it proceeds. And it will still force the store to y to happen sometime **before** this synchronization.

Thread 2 : The x.load is relaxed, so it won't force any synchronization. Even though thread 1 flushed its state to the system, thread 2 does not do anything to make sure it is synchronized with the system. This should mean everything is in an unknown state anarchy. Just because it happens to see a value of 10 doesn't mean it is synchronized with all the things that happened before x.store(10) in thread 1.

Oddly enough, the load of Y will force a synchronization before it proceeds, so from THAT point on things would be what you expect. Messy eh.

Thread 3 : y.load is an acquire, so it will first get whatever state thread 2 has flushed. Unfortunately, the y.store in thread 2 is relaxed, so it hasn't issued any flushing instructions, and may have been moved around by the optimizers. So the results are again highly unpredictable.

Bottom line, mixing modes is dangerous, especially if it involves a relaxed mode. Mixing seq_cst and release/acquire can be done with care, but you really do need to have a thorough understanding of the subtleties. You probably need good debugging tools too!!!

[Prev : Atomic Types](#)

[Next : Implementation Summary](#)

None: Atomic/GCCMM/AtomicSync (last edited 2012-03-05 18:20:39 by PaoloCarlini)